

Modellierung des DRAUSP-Problems als Reinforcement-Learning-Problem

DQN, Action Masking, Monotonicity-aware DQN

Niklas Fritz, Felix Fritz

21.05.2026

- 1 Das DRAUSP-Problem
- 2 Grundlagen: Reinforcement Learning
- 3 Modellierung als RL-Problem
- 4 Das Problem der ungültigen Aktionen
- 5 Gymnasium-Implementierung
- 6 Zusammenfassung & Ausblick

Das DRAUSP-Problem

Dynamic Resource Allocation Under Stochastic Prices (DRAUSP)

Problemstellung

Gegeben T_d Anfragen, die sequenziell eintreffen. Jede Anfrage i besitzt:

- einen **Erlös** $r_i \in \mathbb{R}$
- einen **Kapazitätsbedarf** $q_i \in \mathbb{N}^K$ über K Zeitslots

Entscheidung

Für jede Anfrage muss **sofort und unwiderruflich** entschieden werden:

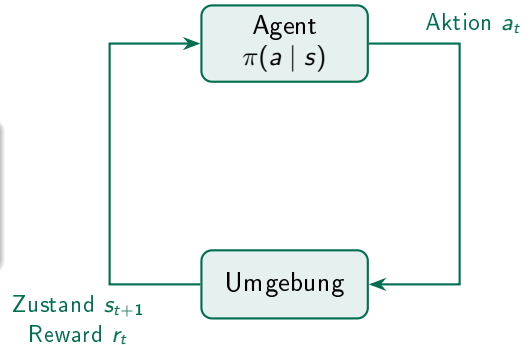
- **Ablehnen** $\rightarrow$ Erlös = 0
- **Akzeptieren ab Slot k** $\rightarrow$ Erlös = r_i , Kapazitäten werden reduziert

Ziel: Maximierung des **Gesamterlöses** unter Einhaltung der Kapazitätsschranken C_k

Reinforcement Learning – Grundprinzip

Kernidee

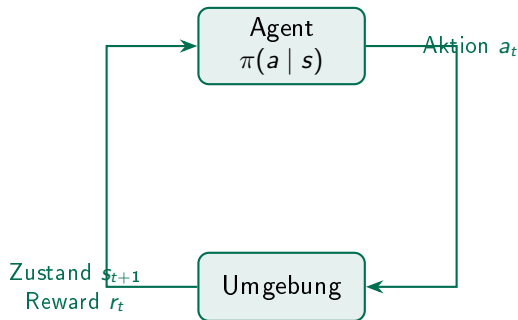
Ein **Agent** interagiert mit einer **Umgebung** und lernt durch Versuch und Irrtum eine optimale **Policy** π .



Zentrale Komponenten:

- **Zustand** $s \in \mathcal{S}$
- **Aktion** $a \in \mathcal{A}$
- **Reward** $r : \mathcal{S} \times \mathcal{A} \rightarrow P(\mathbb{R})$
- **Policy** $\pi(a | s) : \mathcal{S} \rightarrow P(\mathcal{A})$
- **goal: Maximiere den erwarteten kumulierten Reward**

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=1}^{\infty} \gamma^t r_t \right]$$



Q-Learning – Policy aus Q-Werten

Q-Funktion

$Q(s, a)$ gibt den erwarteten kumulierten Reward an, wenn man in Zustand s die Aktion a wählt.

Optimale Policy:

$$\pi^*(s) = \arg \max_{a \in \mathcal{A}} Q^*(s, a)$$

Der Agent wählt stets die Aktion mit dem **höchsten** Q-Wert.

Lernregel (Bellman):

$$Q(s, a) \leftarrow r + \gamma \cdot \max_{a'} Q(s', a')$$

Q-Werte in Zustand s

$$Q(s, a_0) = 12$$

$$Q(s, a_1) = 47 \leftarrow \pi^*$$

$$Q(s, a_2) = 31$$

$$Q(s, a_3) = 5$$

Deep Q-Network (DQN)

Warum ein neuronales Netz?

Zustandsraum zu groß, um $Q(s, a)$ tabellarisch zu speichern ($Q \in \mathbb{R}^{|S| \times |A|}$) $\rightarrow$ ein **neuronales Netz** approximiert $Q(s, a)$ für alle Aktionen gleichzeitig.

$$Q(\cdot | \theta_{DQN}) : s \xrightarrow{\text{DQN}} \begin{pmatrix} Q(s, a_1) \\ Q(s, a_2) \\ \vdots \\ Q(s, a_N) \end{pmatrix}$$

Training des DQN:

- **Loss-Funktion:** **MSE** zwischen Zielwert y_i und Vorhersage $Q(s_i, a_i; \theta^{DQN})$

$$L(\theta^{DQN}) = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i; \theta^{DQN})),$$
$$\theta^{DQN} \leftarrow \theta^{DQN} - \alpha \nabla_{\theta^{DQN}} L(\theta^{DQN})$$

- **Replay Buffer:** Erfahrungen (s, a, r, s') werden gespeichert und zufällig gesampelt
- **Target-Netz:** zweites, seltener aktualisiertes Netz stabilisiert das Training
- **ε -Greedy:** Balance zwischen Exploration und Exploitation
- **Gradient Clipping:** verhindert explodierende Gradienten

Zustandsvektor $s \in \mathcal{S} \subseteq \mathbb{R}^{2K+2}$

$$s = (t, \hat{C}_1, \dots, \hat{C}_K, r, q_1, \dots, q_K)$$

Komponente	Bedeutung	Wertebereich
t	aktueller Zeitschritt	$\{1, \dots, T_d\}$
$\hat{C}_k$	verbleibende Kapazität in Slot k	$\{0, \dots, C_k\}$
r	Erlös der aktuellen Anfrage	$\mathbb{R}_{\geq 0}$
q_k	benötigter Bedarf in Slot k	$\{0, \dots, C_k\}$

Initialzustand:

$$s_1 = (1, C_1, \dots, C_K, r_1, q_1^1, \dots, q_1^K)$$

Aktionsraum und Übergangsfunktion

Aktionsraum $\mathcal{A} = \{0, 1, \dots, K\}$

- **Aktion 0:** Anfrage ablehnen $\rightarrow$ Kapazitäten unverändert
- **Aktion $k > 0$:** Anfrage ab Slot k einplanen $\rightarrow \hat{C}_{k+i} \leftarrow \hat{C}_{k+i} - q_i$ für $i = 0, \dots, \ell_q - 1$

Reward-Funktion

$$R(s, a) = \begin{cases} r & \text{Akzeptieren, Kapazität ausreichend} \\ 0 & \text{Ablehnen} \\ -100 & \text{ungültige Aktion (Kapazitätsverletzung)} \end{cases}$$

Episode endet (terminated = True), wenn:

- alle T_d Anfragen abgearbeitet wurden, **oder**
- eine ungültige Aktion gewählt wurde

Das Problem der ungültigen Aktionen

Problem

Nicht jede Aktion $a \in \mathcal{A}$ ist in jedem Zustand **zulässig**!

Zwei Arten von ungültigen Aktionen:

- ① **Slot zu weit rechts:** $\ell_q + a - 1 > K$

Die Anfrage würde über den letzten Slot hinausragen.

- ② **Kapazitätsverletzung:** $\exists i : \hat{C}_{a+i} < q_i$

Die verbleibende Kapazität reicht nicht aus.

Konsequenz für Standard-DQN

Ein Standard-DQN ohne Behandlung ungültiger Aktionen wählt per $\arg \max$ ggf. eine unzulässige Aktion $\rightarrow$ Reward = -100 $\rightarrow$ **instabiles, langsames Training.**

Idee

Ungültige Aktionen erhalten **vor der Auswahl** einen Q-Wert von $-\infty$, sodass $\arg \max$ sie niemals wählt.

Action Masking im Policy und Target-Netz:

1. Aktionsauswahl

$$\tilde{Q}(s, a) = \begin{cases} Q_{\pi}(s, a) & a \in \mathcal{A}_{\text{valid}}(s) \\ -\infty & \text{sonst} \end{cases}$$

$$a^* = \arg \max_a \tilde{Q}(s, a)$$

Außerdem sampelt man im ϵ -Greedy-Verfahren nur aus den gültigen Aktionen $\mathcal{A}_{\text{valid}}(s)$.

Gymnasium

Standardisiertes Python-Interface für RL-Umgebungen – kompatibel mit gängigen RL-Bibliotheken.

Pflichtmethoden (Gymnasium-Standard):

Methode	Rückgabe	Bedeutung
<code>reset()</code>	<code>obs, info</code>	Neue Episode starten
<code>step(action)</code>	<code>obs, r, terminated, truncated, info</code>	Schritt ausführen
<code>render()</code>	–	Zustand visualisieren

Eigene Erweiterung für Action Masking:

`get_valid_actions()` → `list[int]` Gibt alle zulässigen Aktionen im aktuellen Zustand zurück.

Beitrag

Modellierung des DRAUSP-Problems als Reinforcement-Learning-Problem mit einer standardisierten Gymnasium-Umgebung.

Geplanter Vergleich:

Methode	Typ	Action Masking
Random Policy	Baseline	–
DQN ohne Masking	RL	nein
DQN mit Masking	RL	ja
Monotones DQN	RL (Erweiterung)	ja

1 Allgemeiner Aktionsraum

Bisher: feste Requestgröße ℓ_q . Geplant: variable Requestgrößen

2 Monotone Neural Networks

Architektur erzwingt strukturelle Monotonie in Kapazitäten und Erlös:

$$\hat{C}_k \uparrow \Rightarrow Q(s, a) \uparrow \qquad r \uparrow \Rightarrow Q(s, a) \uparrow$$

3 Evaluation auf Benchmark-Instanzen

- **Konvergenzgeschwindigkeit:** Anzahl Episoden bis zur Stabilisierung
- **Trainings-Stabilität:** Varianz des kumulierten Rewards (Sample-Varianz)
- **Generalisierung:** Out-of-sample Performance auf ungesehenen Instanzen

4 Hyperparameter-Analyse

Exploration-Faktor ϵ , Diskontierungsfaktor γ , Replay-Buffer-Größe, Netzwerkarchitektur

Vielen Dank!

Fragen?